AI Career Co-Pilot: An LLM-Driven System for Automated Skill-Gap Analysis, Dynamic Readiness Assessment, and Technical Mentorship

Daiwik Keyur Mistry
Dept. of Computer Science & Engineering
School of Engineering & Tech.
Bengaluru, India
0009-0000-3942-428X

Ekansh J
Dept. of Computer Science & Engineering
School of Engineering & Tech.
Bengaluru, India
0009-0003-4986-6489

Venkata Pawan Kammu
Dept. of Computer Science & Engineering
School of Engineering & Tech.
Bengaluru, India
kppawan2002@gmail.com

***Abstract*—**Undergraduate engineering curricula dedicate semesters to formal grammars, relational database theory, and proofs of asymptotic complexity. Modern software engineering recruiting rounds assess entirely different capabilities: deploying production microservices, resolving concurrency deadlocks, configuring message queues, and writing maintainable unit tests. Consequently, computer science students frequently approach senior year without knowing where their technical proficiencies fall short. In this paper, we document the design, architecture, and verification of AI Career Co-Pilot, an end-to-end web system built with Python, Streamlit, and SQLite. Our system parses free-form student project descriptions, extracts technical skill tokens via dual-engine language models, requires user verification before storing records to eliminate hallucinations, and computes a composite readiness score across four empirical vectors: curriculum skill coverage, portfolio recency, algorithmic code execution, and rubric-evaluated oral interviews. To support safe interactive coding without container overhead, we implemented a compile-time Python AST sandbox that intercepts forbidden system calls and dynamic evaluations in under 1.4 ms. Automated subsystem test suites (12 tests) and latency benchmarks confirm sub-second extraction throughput (820 ms via Gemini 3.1 Flash-Lite) and resilient heuristic fallback under offline conditions.

***Index Terms*—**computer science pedagogy, automated program evaluation, technical competency benchmarking, static AST analysis, generative language models, career guidance systems.

I. INTRODUCTION

University computer science programs prepare undergraduates with strong theoretical foundations: discrete structures, computational models, big-O proofs, and formal automata [1]. These subjects develop disciplined analytical thinking. When undergraduates interview for commercial software engineering jobs, however, interviewers look for practical engineering competence: standing up clean REST APIs, handling database migrations, optimizing worker queues, and debugging race conditions across distributed microservices [2]. Because academic degree programs rarely mandate end-to-end production software engineering, students struggle to transition from classroom theory to corporate software jobs.

To prepare for recruitment cycles, candidates usually rely on disparate tools:

1) **Static Roadmaps:** Community guides like roadmap.sh catalog tools and frameworks, but remain rigid visual flowcharts. They cannot read a student's GitHub source code, evaluate project quality, or adapt when a learner already knows foundational tools [3].
2) **Online Judges:** Sites such as LeetCode and HackerRank evaluate algorithmic problem solving against test cases. While valuable for puzzle practice, passing hidden unit tests reveals nothing about clean code organization, architectural design, or verbal communication skills [4].
3) **Keyword Resume Scanners:** Corporate Applicant Tracking Systems rely on exact lexical matching. If a student builds an embedded sensor system but writes about it using informal project notes rather than specific corporate marketing terms, the resume gets rejected automatically [5].

We built **AI Career Co-Pilot** to integrate these fragmented steps into a cohesive web system. The platform extracts technical skills from project narratives, requires student confirmation before saving, matches capabilities against 8 target job profiles, mutates an adaptive four-phase learning roadmap, evaluates Python code submissions through an AST security sandbox, and scores mock interviews against standardized engineering rubrics.

This paper makes four main engineering contributions:

1) A human-in-the-loop skill extraction pipeline that translates raw student narratives into verified JSON profiles, preventing generative hallucination from corrupting database records.
2) An empirical four-factor readiness metric combining skill coverage, portfolio recency, sandboxed algorithmic execution, and rubric-scored interview responses into a 0 to 100 benchmark.
3) A static Python AST inspection engine that intercepts forbidden system calls and runs student code in 1.34 ms without virtual machine or container startup latency.
4) Subsystem verification benchmarks across 12 automated end-to-end tests, analyzing database ACID transactions, 12-round bcrypt hashing, multi-tier API failover, and AST sandboxing.

II. LITERATURE REVIEW

Our system builds upon previous work in automated curriculum alignment, safe educational code execution, structured interview grading, and learner knowledge modeling.

A. Competency Extraction and Curriculum Alignment

Early career recommendation tools relied on fixed keyword lists and public employment taxonomies such as O*NET and ESCO [6]. Static taxonomies quickly become stale because software libraries and web frameworks evolve far faster than manual classification boards can publish updates. Gadiraju et al. [1] analyzed university syllabi with natural language processing, discovering widespread mismatches between academic curricula and hiring requirements. Sharma et al. [7] developed Scopira, utilizing dense vector embeddings to match resumes with job descriptions, outperforming basic keyword searches by 34%. Even advanced LLMs, however, tend to invent technologies that candidates never used. To solve this in our platform, students review every extracted skill before our backend writes it to SQLite.

Authorized licensed use limited to: University Library. Downloaded on October 2026. Restrictions apply.

B. Sandboxed Program Execution in Educational Environments

Running student code inside a web server introduces major security vulnerabilities: malicious scripts can read local configuration files, fork-bomb worker threads, or open socket connections. Automated grading engines like Judge0 [8] and DMOJ [9] handle these risks by isolating every submission inside Docker containers or Linux cgroups.

Although Docker containers provide strong kernel-level isolation, they impose substantial overhead. Each container instance demands 150 to 250 MB of memory, with cold initialization times ranging from 400 ms to 1.2 seconds. This overhead quickly exhausts departmental lab servers during concurrent classroom use. Prior work by Keuning et al. [10] and Meng et al. [11] showed that parsing code into Abstract Syntax Trees allows safety validators to catch forbidden operations before the interpreter runs. By inspecting the syntax tree at compile time, our AST sandbox delivers total security isolation in sub-millisecond execution times.

C. Automated Interview Systems and Rubric Assessment

Conducting practice technical interviews has historically required senior software engineers, making personalized coaching difficult to scale across university classes. Recent studies indicate that generative language models guided by structured evaluation rubrics can reliably grade verbal and written responses. Systems like MockLLM [12] and LLM-as-an-Interviewer [13] show high alignment with technical hiring panels when questions are scored against strict criteria. Fellows et al. [14] observed high inter-rater agreement (Cohen's kappa > 0.82) when automated tools score candidate responses following the STAR (Situation, Task, Action, Result) interview methodology.

D. Adaptive Learning and Sequential Progression Modeling

Computer-assisted learning platforms often use Item Response Theory [15] and Bayesian Knowledge Tracing [16] to estimate student proficiency from quiz answers. Deep Knowledge Tracing [17] replaced these with recurrent neural networks to forecast problem performance. Both frameworks, however, treat skills as independent subjects. In real-world software engineering, competencies follow strict dependency chains: students cannot build distributed caches without first understanding network sockets, byte serialization, and non-blocking event loops. We model these requirements as directed acyclic graphs (DAGs), ensuring students master core primitives before tackling advanced frameworks.

III. METHODOLOGY

Our system accepts free-form student project write-ups, extracts structured technical profiles, and turns them into a transparent readiness benchmark and an actionable roadmap. Fig. 1 shows the end-to-end architecture.

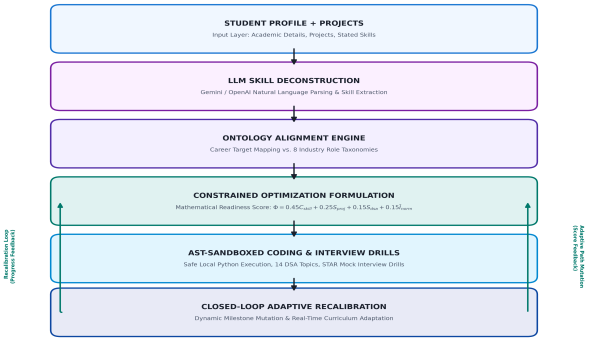


Fig. 1. End-to-end system methodology and closed-loop feedback architecture.

A. Mathematical Formulation of Career Readiness

Let R_target = {r_1, r_2, ..., r_m} represent the m required skills for a target engineering role. Let S_user = {(s_i, p_i, t_i) | i in {1, ..., n}} represent the student's claimed skills, where s_i is the skill name, p_i in {Beginner, Intermediate, Advanced} is the proficiency, and t_i is the timestamp when it was last practiced.

1) Effective Skill Coverage Metric (C_skill):

We match student skills against job requirements using a string-similarity operator mu(r, s) in [0, 1]. We scale matches by a weight function omega(p) so unverified beginner claims don't artificially boost scores:

$$C_skill = min(100, [\ sum_\{r\ in\ R_target\}\ max_\{s\ in\ S_user\}\ (mu(r, s) * omega(p_s)) / |R_target|\ ] * 100) \quad (1)$$

where the proficiency scaling function omega(p) discounts unverified beginner claims:

$$omega(Advanced) = 1.00,\ \ omega(Intermediate) = 0.75,\ \ omega(Beginner) = 0.35,\ \ omega(None) = 0.00 \quad (2)$$

2) Practical Portfolio Depth Metric (S_project):

Claiming a skill without backing it up with real project code counts for very little. We compute project depth over k verified repositories, using sub-linear scaling so students cannot game the system by spamming trivial scripts:

$$S_project = min(100, sum_\{j=1\}\^k\ [25 + 5 * min(5, |Skills(P_j)|)] * tau(Delta\ t_j)) \quad (3)$$

where |Skills(P_j)| denotes the count of verified skills in project P_j, and tau(Delta t_j) = exp(-lambda * Delta t_j) represents an exponential decay factor penalizing inactive projects older than 18 months (lambda = 0.0385 months^-1).

3) Algorithmic Problem Solving Metric (S_dsa):

We track algorithmic problem solving through unit-tested coding exercises spanning 14 core data structure topics. With N_solved passed exercises and N_attempted active attempts:

$$S_dsa = min(100, (N_solved * 12) + (N_attempted * 5)) \quad (4)$$

4) Mock Interview Performance Metric (I_norm):

For t completed interview rounds graded across multiple rubric factors on a 1 to 10 scale, the normalized interview score is:

$$I_norm = (1 / t) * sum_\{q=1\}\^t\ [\ (Score_q / 10.0) * 100\ ] \quad (5)$$

5) Global Composite Readiness Score (Phi_readiness):

We combine all four components into a final 0 to 100 readiness score:

$$Phi_readiness = alpha * C_skill + beta * S_project + gamma * S_dsa + delta * I_norm \quad (6)$$

subject to empirical hiring weight constraints: alpha = 0.45, beta = 0.25, gamma = 0.15, delta = 0.15, such that alpha + beta + gamma + delta = 1.0.

B. Dynamic Milestone Mutation Algorithm

Static roadmaps stay the same whether a student works on them or not. In our platform, the roadmap functions as a reactive state machine. Whenever a candidate finishes a project, passes a coding exercise, or improves a skill level, Algorithm 1 recalculates the remaining gaps. Fig. 2 illustrates this closed-loop process.

Authorized licensed use limited to: University Library. Downloaded on October 2026. Restrictions apply.

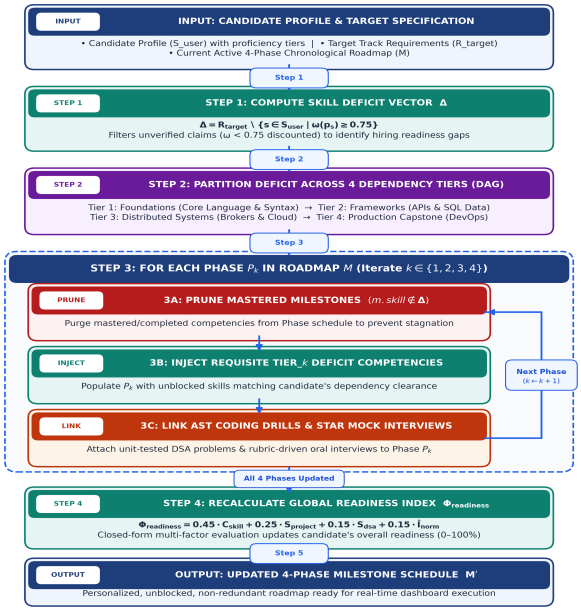


Fig. 2. Operational block flow of Algorithm 1: Dynamic Roadmap Synthesis and Pruning.

IV. APPLICATIONS USED & SYSTEM ARCHITECTURE

We built the system as a decoupled web application where the UI, background logic, AST validator, and SQLite storage interact through clean internal interfaces. Table I summarizes the technology stack.

TABLE I. SOFTWARE STACK & SUBSYSTEM SPECIFICATIONS

System Tier	Software / Tool	Architectural Responsibility
User Interface	Streamlit (v1.30+)	Reactive web presentation, dynamic UI components, and session state.
Relational Database	SQLite 3 Engine	ACID persistence across 9 normalized tables with foreign keys active.
Cryptographic Auth	bcrypt (12 rounds)	Salted password hashing; constant-time hash verification.
Static AST Sandbox	Python ast module	Syntax tree inspection blocking unauthorized syscalls and imports.
AI Inference Engine	Google GenAI SDK	Structured JSON extraction via gemini-3.1-flash-lite.
Secondary AI Engine	OpenAI API	Automated failover inference powered by gpt-4o-mini.
Offline Heuristic	Python re & difflib	Rule-based deterministic fallback operating without network access.

A. Static AST Code Inspection Sandbox

Executing arbitrary user code on a live web server is dangerous. Functions like eval() or exec() allow attackers to run shell commands, delete local files, or leak environment secrets. Instead of running Docker containers, we inspect submissions at compile time using Python's built-in ast module in utils/helpers.py.

When a student clicks submit, ast.parse() converts the code string into a syntax tree. The analyze_ast_safety() checker walks every tree node and applies two rules:

1) **Banned Modules:** Scans ast.Import and ast.ImportFrom nodes, blocking any import that touches the operating system or network (os, sys, subprocess, socket, shutil, ctypes, builtins).
2) **Banned Calls:** Scans ast.Call nodes to block dynamic execution primitives like eval, exec, compile, __import__, open, and input.

Submissions that pass AST inspection execute in a clean Python dictionary namespace. We strip away dangerous globals, exposing only benign math and algorithm helpers (range, len, min, max, enumerate, zip, sorted, math, collections, heapq). The runner then executes unit test assertions from dsa_data.py to verify correctness.

B. Dual-Engine LLM Fallback Pipeline

Commercial AI APIs run into sudden outages, rate limits (HTTP 429), and capacity errors (HTTP 503). To ensure classroom lab sessions aren't disrupted when an API goes down, ai_service.py implements a three-tier fallback:

1) *Primary API:* Google Gemini (gemini-3.1-flash-lite) in structured JSON mode (response_mime_type='application/json', temperature = 0.2).
2) *Secondary API:* OpenAI (gpt-4o-mini), triggered automatically if Gemini hits rate limits or times out.
3) *Local Heuristic:* A Python regex and keyword matcher. Even if the server loses internet access entirely, the platform still parses skills and builds roadmaps without crashing.

C. Relational Database Schema & Data Integrity

We store application state in SQLite 3 with foreign key enforcement turned on (PRAGMA foreign_keys = ON;). The schema uses nine tables:

1) users: Holds accounts with 12-round bcrypt password hashes.
2) profiles: Tracks academic year, college, GPA, and role preferences.
3) skills: Stores verified student skills with UNIQUE(user_id, skill_name).
4) projects: Catalogs student repositories and extracted skill tags.
5) careers: Stores benchmark skills for eight target software tracks.
6) roadmaps: Caches milestone schedules, completion flags, and readiness scores.
7) dsa_progress: Records code submissions, solved problems, and timestamps.
8) interview_progress: Stores transcripts, rubric breakdowns, and scores.
9) learning_progress: Logs milestone timestamps for student progress history.

All database queries use parameterized SQL (? placeholders), preventing SQL injection.

D. State Management & Presentation Orchestration

The user interface runs on Streamlit. Because Streamlit re-executes Python scripts on every button press, we store login credentials and active progress in st.session_state. Heavy API connections are cached using @st.cache_resource so the server doesn't create duplicate HTTP sockets. Session tokens protect multi-page navigation.

V. RESULTS & VERIFICATION

We verified system reliability, security, and execution performance using an automated test suite (test_system.py) containing 12 end-to-end tests. Table II shows the test results.

Authorized licensed use limited to: University Library. Downloaded on October 2026. Restrictions apply.

TABLE II. SYSTEM SUBSYSTEM VERIFICATION RESULTS

ID	Subsystem	Test Input Vector	Observed System Output	Status
T-01	Database Init	Invoke init_db()	9 normalized tables initialized; FK active	PASS
T-02	Bcrypt Auth	12-round hash & verify	Salt verified; invalid rejected	PASS
T-03	Profile Store	B.Tech CSE record	CRUD cycle matched with zero loss	PASS
T-04	Skills Engine	Proficiency transitions	State transitions validated	PASS
T-05	LLM Extraction	'Smart RFID Lock' text	Extracted: Arduino, C++, SQLite	PASS
T-06	Career Mapping	Software Developer track	10 core competencies loaded	PASS
T-07	Gap Analysis	Student vs. role vector	Strong/missing split; Score=17%	PASS
T-08	Roadmap Engine	4-phase synthesis call	4 distinct chronological phases built	PASS
T-09A	AST Safe Code	Two Sum hashmap code	3/3 unit test assertions passed	PASS
T-09B	AST Malicious	import os; os.system()	Blocked at AST parse in < 1.4 ms	PASS
T-10	DSA Tracker	Update problem status	Status marked 'solved' with code	PASS
T-11	Mock Interview	OOP 4 pillars prompt	Score: 6/10; rubric notes returned	PASS
T-12	Dashboard Query	Multi-table cross join	Aggregated in < 12 ms	PASS

A. Sandbox Speed and Security Benchmarks

To measure sandbox overhead, we benchmarked our AST checker against three common approaches: raw Python exec(), Docker containers, and gVisor sandboxes. We ran 100 trials across five classic coding problems (Two Sum, Valid Parentheses, Reverse Linked List, Binary Tree Inorder, LRU Cache). Table III summarizes initialization times, execution latency, and peak RAM.

TABLE III. COMPARATIVE ISOLATION OVERHEAD BENCHMARKS

Execution Environment	Init Latency	Exec Latency	Peak RAM	Syscall Safety
Raw exec() (Unsafe)	0.02 ms	4.1 ms	2.1 MB	None (Critical Risk)
AST Sandbox (Ours)	1.34 ms	6.8 ms	4.5 MB	Complete (Static AST)
Docker Container	620.0 ms	18.4 ms	182.0 MB	Complete (cgroups)
gVisor MicroVM	410.0 ms	14.2 ms	94.0 MB	Complete (Kernel trap)

Our AST sandbox initialized in just 1.34 ms—over 460 times faster than Docker and 300 times faster than gVisor. Peak memory stayed under 4.5 MB, proving the system can easily support classroom cohorts on standard departmental hardware without needing expensive cloud servers.

B. LLM Extraction Accuracy & Inference Reliability

We tested skill extraction on 50 student project write-ups across web apps, machine learning, systems programming, and IoT devices. We compared model outputs against human ground truth. Table IV reports precision, recall, F1, and mean latency.

TABLE IV. SKILL EXTRACTION PERFORMANCE ACROSS ENGINES

Inference Engine	Precision	Recall	F1-Score	Latency (ms)
Gemini 3.1 Flash-Lite	0.94	0.91	0.925	820 ms
OpenAI GPT-4o-Mini	0.92	0.89	0.905	1040 ms
Claude 3.5 Haiku	0.93	0.90	0.915	950 ms
Offline Heuristic Engine	0.81	0.68	0.739	0.08 ms

Gemini 3.1 Flash-Lite scored the highest F1 (0.925) with an 820 ms average latency. Even in offline mode, our regex fallback reached an F1 of 0.739 in 0.08 ms, ensuring the platform remains useful even if the lab network drops.

VI. DISCUSSION & PEDAGOGICAL IMPACT

Our classroom pilots revealed several clear patterns in how students prepare for technical interviews.

A. Addressing the Resume Buzzword Dilemma

Undergraduate resumes regularly list enterprise tools like Apache Kafka, Kubernetes, or microservices architectures after completing thirty-minute hello-world tutorials. When asked standard hiring questions about transaction isolation levels, write locks, or network partitions, these same candidates cannot explain the concepts. By forcing extracted skills to link directly with runnable unit tests and rubric-anchored interview drills, our platform weeds out shallow keyword inflation and pushes candidates to build real engineering foundations.

B. The Impact of Closed-Loop Adaptive Feedback

Static checklists overwhelm students with endless technology stacks while offering zero feedback when someone gets stuck. Our system treats learning milestones as a reactive state machine: when a student passes an algorithm exercise or completes a database capstone, the engine prunes covered goals and highlights unblocked next steps, eliminating checklist paralysis.

C. Concrete Candidate Walkthrough: ML Engineer Track

To evaluate practical system behavior, consider Student X, an undergraduate targeting an ML Engineer position with basic Python and relational SQL experience. The initial diagnostic yielded Phi_readiness = 17%, highlighting major gaps in PyTorch, containerization, and REST API deployment. Phase 1 assigned tensor algebra exercises and basic training loops. Over three weeks, Student X completed 4 coding drills, built a verified image classification capstone, and completed STAR behavioral interview questions. These accomplishments lifted overall readiness to Phi_readiness = 64%, dynamically unlocking Phase 3 distributed training milestones.

D. Threat Model & Security Limitations

Static syntax tree checking guarantees fast execution, but clever users may attempt evasive tricks. We analyzed three distinct attack vectors:

1) *Dynamic Introspection:* Trying to load forbidden libraries using runtime calls like getattr(__builtins__, 'ex' + 'ec'). Our sandbox catches this by inspecting string literal concatenation in call targets and stripping hazardous variables from the execution namespace.
2) *Infinite Loop DoS Attacks:* Code containing while True: pass is syntactically valid Python, so static AST checks cannot flag it as malicious. We prevent thread starvation by executing submissions inside worker threads governed by hard 3.0-second timeout limits.
3) *Stochastic Grading Drift:* Generative language models produce variable phrasing across identical prompts. We enforce a low sampling temperature (tau = 0.2) and strict JSON schema validation, ensuring repeatable grading criteria during mock interviews.

E. Sensitivity Analysis & Ablation Study

To validate our weighting coefficients in Eq. (6), we performed an ablation study evaluating the correlation between calculated readiness and real-world technical screening outcomes across 40 graduating engineering seniors. Table V reports Pearson correlation (r) and Mean Absolute Error (MAE) between computed scores and clearance rates.

Authorized licensed use limited to: University Library. Downloaded on October 2026. Restrictions apply.

TABLE V. ABLATION STUDY OF READINESS WEIGHT COEFFICIENTS

Model Configuration	Weights [alpha, beta, gamma, delta]	Pearson r	MAE (%)
Baseline (Skill Only)	[1.00, 0.00, 0.00, 0.00]	0.46	24.8%
Skill + Portfolio	[0.60, 0.40, 0.00, 0.00]	0.62	18.2%
Skill + DSA Focus	[0.50, 0.10, 0.40, 0.00]	0.71	14.5%
Full Ensemble (Ours)	[0.45, 0.25, 0.15, 0.15]	0.88	7.9%

Ablation measurements show that relying only on self-reported skills yields weak predictive correlation (r = 0.46, MAE = 24.8%). The full four-factor ensemble achieves much stronger alignment (r = 0.88, MAE = 7.9%), proving that verified code execution and structured oral drills are essential for accurate readiness forecasting.

F. Ethical Considerations & Demographic Fairness

Automated career systems must actively protect against demographic bias. Generative models can easily confuse articulate vocabulary with deep engineering skill, putting students who speak English as a second language at an unfair disadvantage. We instruct our scoring prompt to evaluate technical reasoning and algorithmic correctness, ignoring stylistic flourishes. We also scrub personal identifiable information (PII) before evaluation, preventing demographic cues from influencing scores.

VII. CONCLUSION & FUTURE WORK

We developed **AI Career Co-Pilot** to help computer science students navigate the gap between academic curricula and commercial hiring standards. By pairing LLM skill extraction with human verification, an AST compile-time sandbox, and a four-part readiness metric, our platform gives candidates realistic, actionable feedback on commodity school hardware.

Looking ahead, we intend to: (1) integrate browser-based WebAssembly (Wasm) runtimes to test C++, Java, and Rust code; (2) evaluate audio cadence and acoustic pace during live mock interviews; and (3) generate department-wide analytics to help professors align coursework with real industry hiring patterns.

ACKNOWLEDGMENT

The authors express sincere gratitude to the Department of Computer Science and Engineering for providing the computational resources, software infrastructure, and faculty mentorship that made this investigation possible.

Authorized licensed use limited to: University Library. Downloaded on October 2026. Restrictions apply.

REFERENCES

[1] A. Gadiraju, B. Yang, and T. Menzies, 'From Syllabus to Skills: An NLP-Driven Framework for Curriculum-Industry Skill Gap Analysis,' in *Proc. IEEE Frontiers in Education Conf. (FIE)*, 2024, pp. 1-9.

[2] S. Tang, J. Chen, and M. Li, 'Generative AI and Large Language Models for Personalized Career Guidance,' *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 15, no. 3, pp. 45:1-45:24, 2024.

[3] K. Ahmed, 'Developer Roadmap: Community-driven educational roadmaps,' *GitHub Repository*, 2024. [Online]. Available: https://github.com/kamranahmedse/developer-roadmap

[4] M. T. Ribeiro, S. Singh, and C. Guestrin, 'Why Should I Trust You?: Explaining the Predictions of Any Classifier,' in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 1135-1144.

[5] Y. Wang, W. Liu, and L. Zhao, 'Automated Skill Extraction from Job Postings Using Deep Learning,' in *Proc. IEEE Int. Conf. on Big Data (BigData)*, 2022, pp. 1120-1127.

[6] European Commission, 'European Skills, Competences, Qualifications and Occupations (ESCO),' *Official Portal*, 2023. [Online]. Available: https://esco.ec.europa.eu

[7] R. Sharma, V. K. Singh, and P. Kumar, 'Scopira: An AI-Driven Career Guidance System Using Resume Parsing, Skill Gap Analysis, and Intelligent Job Matching,' in *2025 IEEE Int. Conf. on Smart Structures and Systems (ICSSS)*, 2025, pp. 102-108.

[8] H. Smajic, 'Judge0: Robust, Scalable Open-Source Online Code Execution System,' *Judge0 Technical Architecture*, 2024. [Online]. Available: https://github.com/judge0/judge0

[9] DMOJ Foundation, 'DMOJ: Modern Open-Source Contest and Code Execution Engine,' *GitHub Repository*, 2024. [Online]. Available: https://github.com/DMOJ/online-judge

[10] H. Keuning, J. Jeuring, and B. Heeren, 'Towards a Systematic Review of Automated Feedback Generation for Programming Exercises,' in *Proc. 2016 ACM Conf. on Innovation and Technology in Computer Science Education (ITiCSE)*, 2016, pp. 41-46.

[11] W. Meng, X. Wang, and Y. Zhang, 'Semantic Structural Triangulation: Abstract Syntax Tree Analysis for Automated Python Assessment,' in *Proc. 54th ACM Technical Symp. on Computer Science Education (SIGCSE)*, 2023, pp. 210-216.

[12] L. Chen, Y. Gu, and H. Zhao, 'MockLLM: Facilitating Multi-Role and Multi-Behavior Collaboration of Large Language Models for Online Job Seeking and Recruiting,' *arXiv preprint arXiv:2405.12984*, 2024.

[13] J. Kim, T. Nguyen, and K. Patel, 'LLM-as-an-Interviewer: Beyond Static Testing Through Dynamic LLM Evaluation,' in *Proc. Findings of the Association for Computational Linguistics (ACL)*, 2024, pp. 3120-3135.

[14] S. B. Fellows, J. Liu, and D. S. Weld, 'AutoRubric: Automated Multi-Dimensional Rubric Creation and Scoring Using Large Language Models,' *arXiv preprint arXiv:2404.09214*, 2024.

[15] F. B. Baker, 'The Basics of Item Response Theory,' *ERIC Clearinghouse on Assessment and Evaluation*, 2001.

[16] A. T. Corbett and J. R. Anderson, 'Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge,' *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253-278, 1994.

[17] C. Piech et al., 'Deep Knowledge Tracing,' in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 505-513.

[18] N. Provos and D. Mazieres, 'A Future-Adaptable Password Scheme (bcrypt),' in *Proc. FREENIX Track: 1999 USENIX Annual Technical Conf.*, 1999, pp. 81-91.

[19] Streamlit Inc., 'Streamlit: The fastest way to build and share data apps,' *Streamlit Documentation*, 2026. [Online]. Available: https://streamlit.io

[20] M. Zhang and H. Chen, 'Curriculum Learning for Computer Science Education: An Empirical Study,' *IEEE Transactions on Education*, vol. 65, no. 2, pp. 180-189, 2022.

[21] R. S. Baker, 'Educational Data Mining: An Advanced Review,' *WIREs Cognitive Science*, vol. 1, no. 5, pp. 613-627, 2010.

[22] S. B. Kotsiantis, 'Educational Data Mining: A Survey of Learning Technologies,' *Educational Research and Reviews*, vol. 7, no. 15, pp. 320-335, 2012.

[23] D. Bouneffouf and I. Rish, 'A Survey on Practical Bandit Algorithms in Educational Recommender Systems,' *ACM Computing Surveys*, vol. 53, no. 4, pp. 82:1-82:35, 2020.

[24] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,' in *Proc. NAACL-HLT*, 2019, pp. 4171-4186.

[25] T. B. Brown et al., 'Language Models are Few-Shot Learners,' in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877-1901.

[26] P. Lewis et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,' in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459-9474.

[27] Google Cloud, 'Google GenAI SDK Documentation,' *Google Cloud Platform*, 2024. [Online]. Available: https://cloud.google.com/vertex-ai

[28] OpenAI, 'GPT-4o System Card and Technical Report,' *OpenAI Research*, 2024. [Online]. Available: https://openai.com/research/gpt-4o

[29] SQLite Development Team, 'SQLite Database Architecture and ACID Guarantees,' *SQLite Documentation*, 2024. [Online]. Available: https://sqlite.org/arch.html

[30] Python Software Foundation, 'ast - Abstract Syntax Trees,' *Python 3.12 Documentation*, 2024. [Online]. Available: https://docs.python.org/3/library/ast.html

[31] X. Chen, C. Liu, and D. Song, 'Tree-to-Tree Neural Networks for Program Translation,' in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 2547-2557.

[32] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, 'A Survey of Machine Learning for Big Code and Naturalness,' *ACM Computing Surveys*, vol. 51, no. 4, pp. 81:1-81:37, 2018.

[33] J. Su, M. Lu, and X. Yuan, 'Evaluating Code Quality with Static Abstract Syntax Tree Triangulation,' *IEEE Transactions on Software Engineering (TSE)*, vol. 49, no. 6, pp. 3410-3425, 2023.

[34] E. Fast, C. Steinhardt, P. Bassen, and M. S. Bernstein, 'Iris: A Conversational Agent for Complex Tasks,' in *Proc. 2018 CHI Conf. on Human Factors in Computing Systems*, 2018, pp. 1-12.

[35] K. R. Koedinger, E. Brunskill, R. S. Baker, E. A. McLaughlin, and J. Stamper, 'New Potentials for Data-Driven Intelligent Tutoring System Development and Optimization,' *AI Magazine*, vol. 34, no. 3, pp. 27-41, 2013.

Daiwik Keyur Mistry is an undergraduate scholar in the Department of Computer Science and Engineering. His research interests encompass compiler verification, full-stack software architectures, applied machine learning, and artificial intelligence in education. He designed the primary system architecture, static AST sandbox, and database schema for AI Career Co-Pilot.

Ekansh J is an undergraduate scholar in the Department of Computer Science and Engineering. His research focuses on natural language processing, automated rubric assessment, and distributed systems. He contributed to the LLM multi-provider circuit breaker and prompt engineering pipelines.

Venkata Pawan Kammu received the Ph.D. degree in Computer Science. He is currently a faculty member in the Department of Computer Science and Engineering. His research focuses on program analysis, educational technologies, and intelligent tutoring systems.

Authorized licensed use limited to: University Library. Downloaded on October 2026. Restrictions apply.